



Trabalho Final

Shell Script

Instituto Federal Baiano — *campus* Guanambi

Professor: Prof. Reinaldo Cotrim

Orientações

Em todos os programas, o aluno deverá:

- verificar se a memória foi alocada corretamente;
- apresentar uma mensagem de erro quando a alocação falhar;
- evitar acessos fora dos limites das áreas alocadas;
- utilizar `free()` para liberar a memória antes do encerramento;
- compilar os códigos utilizando as opções `-Wall` e `-Wextra`.

Questões

1. Leitura de um vetor dinâmico

Escreva um programa que leia a quantidade n de números inteiros. Utilize `malloc()` para alocar um vetor com n posições, leia os elementos e exiba-os na mesma ordem em que foram digitados.

2. Maior e menor elemento

Aloque dinamicamente um vetor de n números inteiros. Depois da leitura, encontre e exiba o maior e o menor elemento, juntamente com suas respectivas posições no vetor.

3. Elementos acima da média

Utilize `malloc()` para criar um vetor de n números reais. Leia os valores, calcule a média dos elementos e exiba somente os valores que estiverem acima da média.

4. Inversão de um vetor

Leia n números inteiros em um vetor alocado dinamicamente. Inverta a ordem dos elementos no próprio vetor, sem criar um segundo vetor, e apresente o resultado.

5. Separação de números pares e ímpares

Leia n números inteiros em um vetor dinâmico. Em seguida, conte a quantidade de valores pares e ímpares e aloque dois novos vetores com os tamanhos exatos necessários.

Copie os valores para os vetores correspondentes e exiba os números pares e ímpares separadamente.

6. Redimensionamento com `realloc()`

Aloque inicialmente um vetor para cinco números inteiros. Depois da leitura, pergunte quantos números o usuário deseja acrescentar. Redimensione o vetor utilizando `realloc()`, leia os novos elementos e exiba o vetor completo.

O programa deverá preservar o endereço original caso o `realloc()` falhe, evitando a perda da área de memória anteriormente alocada.

7. Remoção de um elemento

Aloque dinamicamente um vetor de n números inteiros. Solicite ao usuário uma posição a ser removida, desloque os elementos seguintes uma posição para a esquerda e reduza o tamanho do vetor utilizando `realloc()`.

O programa deverá verificar se a posição informada é válida.

8. Matriz alocada dinamicamente

Leia a quantidade de linhas e colunas de uma matriz de números inteiros. Aloque a matriz dinamicamente utilizando um vetor de ponteiros, leia seus elementos e realize as seguintes operações:

- exibir a matriz completa;
- calcular a soma de todos os elementos;
- calcular e exibir a soma de cada linha;
- encontrar o maior elemento da matriz;
- liberar cada linha e, posteriormente, o vetor de ponteiros.

9. Comparação entre `malloc()` e `calloc()`

Desenvolva duas versões de um programa que aloque um vetor grande, por exemplo, com 100 000 000 de números inteiros:

- **Versão A:** utilize `malloc()` e preencha todas as posições com zero;
- **Versão B:** utilize `calloc()`, aproveitando a inicialização automática das posições com zero.

Em ambas as versões, percorra o vetor e calcule a soma dos elementos. Exiba a soma para garantir que a memória alocada foi realmente acessada.

Compile os programas sem otimização:

```
gcc -O0 -Wall -Wextra malloc_teste.c -o malloc_teste
gcc -O0 -Wall -Wextra calloc_teste.c -o calloc_teste
```

Meça o tempo e o consumo de memória:

```
/usr/bin/time -v ./malloc_teste
/usr/bin/time -v ./calloc_teste
```

Execute cada programa pelo menos cinco vezes. Registre o tempo real, o tempo de usuário, o tempo de sistema e a memória máxima utilizada. Calcule a média dos tempos obtidos e explique qual versão apresentou melhor desempenho.

10. Comparação quando o vetor será sobrescrito

Crie duas versões de um programa que aloquem um vetor grande de números inteiros:

- na primeira versão, utilize `malloc()` e preencha o vetor com os valores de 0 até $n - 1$;
- na segunda versão, utilize `calloc()` e depois preencha o vetor com os mesmos valores.

Calcule e exiba a soma dos elementos nas duas versões. Utilize o comando a seguir para medir a execução de cada programa:

```
/usr/bin/time -v ./programa
```

Execute cada versão pelo menos cinco vezes e responda:

- A inicialização automática do `calloc()` trouxe alguma vantagem nesta situação?
- Qual versão apresentou o menor tempo médio de execução?
- Houve diferença significativa no consumo de memória?
- Por que o `calloc()` pode não ser mais rápido quando todas as posições serão sobrescritas?

11. Tratamento de falha na alocação de memória

Implemente um programa que tente alocar memória para 50 000 000 de números inteiros, aproximadamente 200 MB, utilizando `malloc()`.

O programa deverá:

- informar a quantidade de bytes que tentará alocar;
- verificar se `malloc()` retornou `NULL`;
- apresentar uma mensagem quando a alocação falhar;
- encerrar com `EXIT_FAILURE` em caso de erro;
- acessar a área de memória somente quando a alocação for bem-sucedida;
- liberar a memória com `free()` antes do encerramento.

Compile o programa:

```
gcc -Wall -Wextra falha_alocacao.c -o falha_alocacao
```

No Linux, utilize o Bash para limitar a memória virtual disponível para o processo a aproximadamente 100 MB:

```
(ulimit -v 100000; ./falha_alocacao)
```

Depois, execute o programa novamente sem a limitação:

```
./falha_alocacao
```

Compare as execuções e responda:

- O que o valor `NULL` retornado por `malloc()` representa?
- Por que o programa não deve acessar o vetor quando a alocação falha?
- Qual é a importância de verificar o retorno das funções de alocação?
- Qual é a função de `EXIT_FAILURE`?

A limitação com `ulimit` deverá ser aplicada somente ao processo do programa, evitando a tentativa de esgotar toda a memória do computador.